



Phage
SECURITY

Robin

Security Review

Conducted by:

Pyro, Security Researcher

YanecaB, Junior Security Researcher

14.09.2025



Table of Contents

Disclaimer	3
System overview	3
Executive summary	3
Overview	3
Timeline	3
Scope	4
Issues Found	4
Findings	5
High Severity	5
[H-01] DoS in RobinStakingVault::_unlockYield via token donation	5
[H-02] 'createVault' can be used to brick markets	6
Low Severity	7
[L-01] Contracts are upgradable, but lack a storage gap	7
[L-02] Use safeIncreaseAllowance instead of raw approve	8



Disclaimer

Audits are a time, resource, and expertise-bound effort where trained experts evaluate smart contracts using a combination of automated and manual techniques to identify as many vulnerabilities as possible. Audits can reveal the presence of vulnerabilities **but cannot guarantee their absence**.

System overview

Robin is a yield-farming protocol that lets users stake Polymarket outcome tokens (YES/NO) into isolated vaults per prediction market. The system automatically pairs and merges tokens to USDC, supplies it to Aave for lending yield, then distributes rewards via time-weighted scoring after market resolution—essentially turning prediction market positions into passive income generators.

Executive summary

Overview

Project Name	Robin Vaults
Repository	private
Commit hash	9dbf12383eca67174a87eac00cd76a66d9559126
Remediation	7b8e57a4ff1a836a6b1ec47e053dc1e741eb793d
Methods	Manual review

Timeline

Audit kick-off	08.09.2025
End of audit	12.09.2025
Remediations start	13.09.2025
Remediations end	13.09.2025



Scope

src/AaveStakingVault.sol
src/PolymarketAaveStakingVault.sol
src/PolymarketStakingVault.sol
src/RobinStakingVault.sol
src/RobinVaultManager.sol
src/TimeWeighedScorer.sol
src/VaultPausable.sol

Issues Found

Severity	Count
High	2
Medium	0
Low	2



Findings

High Severity

[H-01] DoS in RobinStakingVault::_unlockYield via token donation

The `_pmRedeemWinningToUsd()` function in `PolymarketStakingVault` can be DoSed for markets where `negRisk` is `true`. The function calculates the amount of redeemed USDC by taking a balance snapshot of the underlying `Usd` token before and after the redemption calls.

```
uint256 beforeBal = underlyingUsd.balanceOf(address(this)); //<----

uint256[] memory indexSets = new uint256[](winningPosition ==
    WinningPosition.BOTH ? 2 : 1);
if (winningPosition == WinningPosition.YES) {
    indexSets[0] = YES_INDEX_SET;
} else if (winningPosition == WinningPosition.NO) {
    indexSets[0] = NO_INDEX_SET;
} else if (winningPosition == WinningPosition.BOTH) {
    indexSets[0] = YES_INDEX_SET;
    indexSets[1] = NO_INDEX_SET;
}
ctf.redeemPositions(address(polymarketCollateral),
    PARENT_COLLECTION_ID, conditionId, indexSets);

//If market uses WCOL, we have to unwrap the WCOL into USDC after
// redemption.
if (negRisk) {
    uint256 wcolBal = IWcol(address(polymarketCollateral)).balanceOf(
        address(this));
    IWcol(address(polymarketCollateral)).unwrap(address(this),
        wcolBal); //<---
}
uint256 afterBal = underlyingUsd.balanceOf(address(this)); //<---
redeemedUsd = afterBal - beforeBal;
return redeemedUsd;
```

For `negRisk` markets, the collateral is WCOL. After redeeming the Polymarket outcome tokens for WCOL, the function proceeds to unwrap the entire WCOL balance of the contract into USDC.

An attacker can exploit this by sending a small amount of WCOL tokens directly to the contract address before `_unlockYield` is called. When `_pmRedeemWinningToUsd` is executed, these donated WCOL tokens will also be unwrapped, inflating the calculated `redeemedUsd`.

The calling function, `_unlockYield` in `RobinStakingVault`, has a strict check:



```
uint256 winLeft = winningPosition == WinningPosition.YES ? unpairedYes :
unpairedNo;
if (winningPosition == WinningPosition.BOTH) {
    winLeft = (unpairedYes + unpairedNo) / 2;
}
if (winLeft > 0) {
    uint256 got = _pmRedeemWinningToUsd();
    if (got != _pmUsdAmountForOutcome(winLeft)) revert RedeemMismatch(
    ); //<---
```

The inflated got amount will not match the expected winLeft amount, causing the transaction to revert, which stops unlocking the yield.

Recommendation

Update RobinStakingVault::_unlockYield redeem check to be:

```
if (winLeft > 0) {
    uint256 got = _pmRedeemWinningToUsd();
-   if (got != _pmUsdAmountForOutcome(winLeft)) revert RedeemMismatch();
+   if (got < _pmUsdAmountForOutcome(winLeft)) revert RedeemMismatch();
```

Resolution: Fixed in 3015cc3

[H-02] 'createVault' can be used to brick markets

createVault is permissionless and allows anyone to create a market for a given conditionId, where there can be only one market per ID.

```
function createVault(bytes32 conditionId, bool negRisk, address collateral)
    external whenNotPaused returns (address vault) {
    if (vaultOf[conditionId] != address(0)) revert VaultExists(conditionId,
        vaultOf[conditionId]);
```

The function also allows the market to use the NegRisk adapter. However, an issue arises when a market that does not have a NegRisk adapter is set to use one. collateral also needs to be set to polymarketWcol so the bellow if is avoided.

```
function createVault(bytes32 conditionId, bool negRisk, address collateral)
    external whenNotPaused returns (address vault) {
    if (negRisk && collateral != polymarketWcol) revert InvalidCollateral(
        collateral);
```



When this happens, all of the position IDs become corrupted and the market becomes inoperable.

```
bytes32 yesColl = ctf.getCollectionId(PARENT_COLLECTION_ID,
    conditionId, YES_INDEX_SET);
bytes32 noColl = ctf.getCollectionId(PARENT_COLLECTION_ID, conditionId,
    NO_INDEX_SET);
yesPositionId = ctf.getPositionId(address(polymarketCollateral),
    yesColl);
noPositionId = ctf.getPositionId(address(polymarketCollateral), noColl);
```

The market will exist (even though it will not function) and will prevent the creation of a new one for the same conditionId.

Recommendation

Consider adding `onlyOwner` to `createVault`. Alternatively, if vault creation must remain public, consider hashing `conditionId`, `negRisk`, and `collateral` together and using that as the vault ID instead of `vaultOf[conditionId]`.

Resolution: Fixed in 7b8e57a

Low Severity

[L-01] Contracts are upgradable, but lack a storage gap

If any future upgrades add storage variables to any of the inherited contracts (basically all but `PolymarketAaveStakingVault`), the storage slots for the contracts inheriting from the upgrading contracts will shift, causing a storage collision and bricking the contracts.

Recommendation

The standard practice is to add `uint256[50] private __gap;` to reserve 50 storage slots. When adding new storage slots during upgrades, subtract from that amount accordingly.

Resolution: Fixed in 289a8f3



[L-02] Use `safeIncreaseAllowance` instead of `raw approve`

The current initialization in `AaveStakingVault` and `PolymarketStakingVault` uses `IERC20.approve` directly. If `underlyingUsd` is set to `USDT` (a commonly used USD stablecoin), the transaction will revert because `USDT`'s `approve` does not return a boolean as expected by the ERC-20 interface.

Recommendation

Replace `raw approve` calls with OpenZeppelin's `SafeERC20.safeIncreaseAllowance` to handle non-standard ERC-20 tokens like `USDT` safely.

Resolution: Fixed in 4406e47